CS 6158 Midterm Progress Report

Budget-Constrained Selective Runtime Monitoring

Shiyun Cheng and Gengkun Shao · October 19, 2026

1. Research Goal and Progress

Our project asks how to allocate a fixed budget among runtime monitor-creation locations using historical execution frequency, cost, useful traces, violation reports, and redundancy. The primary research question is whether budget-aware selection preserves more violations than simple heuristics or *Valg*, an adaptive online monitor-creation policy, at comparable *realized* overhead.

At midterm, we have implemented five selection baselines and a reproducible evaluation pipeline, and performed offline selection experiments on historical TraceMOP monitor-profile data for two real Java projects. Separately, we executed a small-scale Valg artifact reproduction comparing ValgJ and JavaMOP on four Java projects, with one complete ValgT/TraceMOP pair. These are distinct experiments: our budgeted policies have not yet been deployed in the Valg runtime or compared against Valg at matched actual cost.

2. Implemented Modules and Methods

The offline pipeline produces selected group IDs, budget usage, recall metrics, CSV tables, and plots. It implements random feasible selection (30 draws per budget), frequency ranking, standalone utility/cost ranking, redundancy-aware greedy selection, and additive knapsack. The committed runner reads the extracted CSV profile directly, so the reported analysis can be rerun without a project-specific spreadsheet package.

For candidate group *i*, historical trace set T_i, selected set S, and proxy cost c_i, the current experiment approximates max $|\cup_{i \in S} T_i|$ subject to $\sum_{i \in S} c_i \le B$. Trace recall is measured against the union of trace IDs already present in the historical workbook. The proxy is an event-check count, not time.

Candidate rows are inferred location-by-specification groups, not verified independently switchable monitor-creation sites. Real-trace violation weighting is not yet supported because the profile lacks reliable location-attributed violation sets.

3. Historical Real-Trace Offline Evaluation

We processed javadbf (153 candidate groups, 278 observed distinct trace IDs) and jlemmagen (66 groups, 2,115 trace IDs). Budgets were 10%, 20%, 30%, 50%, and 80% of summed event-check proxies. The following table reports the 20% proxy-budget result.

Strategy	javadbf	jlemmagen
Random (30-trial mean)	86.51%	65.56%
Frequency	44.96%	88.84%
Utility/cost	97.84%	99.67%
Redundancy-aware greedy	97.84%	99.67%
Additive knapsack	89.57%	99.57%

The cost-aware rankings preserved most historical distinct traces at this proxy budget. Utility/cost and greedy tied on both workloads, so the result does not establish a benefit from explicitly modeling redundancy. Additive knapsack can perform worse because its objective double-counts overlapping traces. Frequency selections can be non-monotone because each budget independently rescans a fixed ordering and admits any group that fits.

4. Partial Valg Artifact Reproduction

We used the official Valg artifact scripts in a prepared Docker container and pinned project commits. Every project completed no-RV, ValgJ, and JavaMOP. Each value below is one end-to-end measurement; the table does not estimate timing variance or monitoring-only time.

Project	no-RV (s)	ValgJ (s)	JavaMOP (s)	Reduction
xeger	10.428	44.714	66.003	32.3%
javadbf	11.142	61.079	38.795	-57.4%
route53-infima	13.673	41.806	59.767	30.1%
jlemmagen	9.389	114.502	315.707	63.7%

ValgJ was faster on three of four projects, with a median reduction of 31.2%. Each ValgJ/JavaMOP pair reported the same number of distinct violation rows, although ValgJ reported fewer violation occurrences. We describe this as preservation of the observed violation signatures, not general violation recall.

Only jlemmagen completed both trace modes. ValgT took 184.600 s versus 349.056 s for TraceMOP (47.1% reduction), retained 1,840 of 2,115 unique traces (87.0%), and reduced derived duplicate trace instances from 1,258,457 to 79,812 (93.7%).

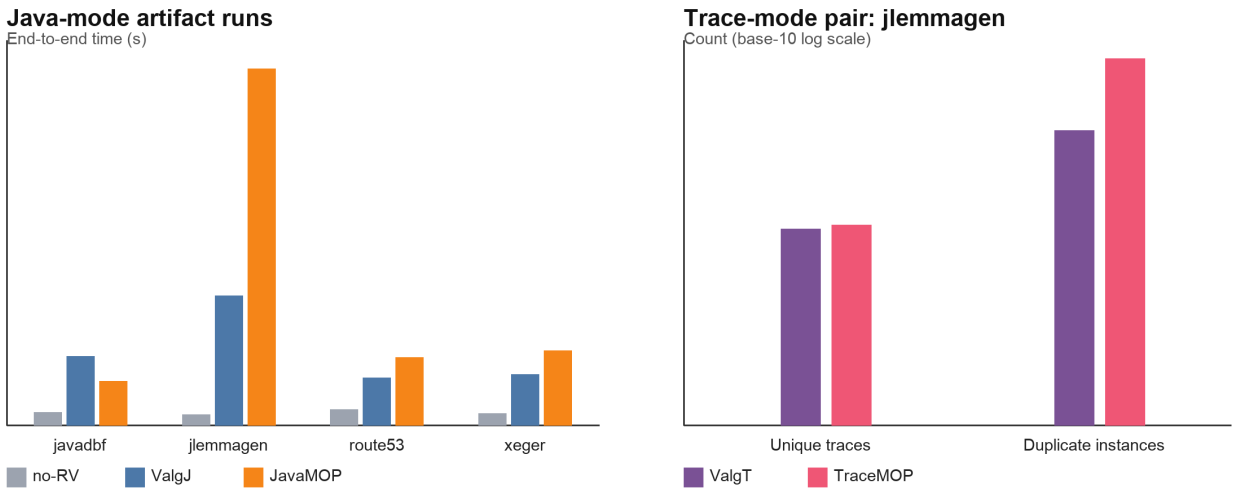


Figure 1. Subset reproduction: Java-mode end-to-end time and the one complete trace-mode pair.

These runs validate the local artifact environment and provide concrete baselines. They cannot yet be directly compared with offline proxy-budget choices because the two experiments use different cost definitions and no matched realized overhead.

5. Challenges, Risks, and What We Learned

Violation attribution. Historical reports identify project-level violations but not whether each candidate location would independently detect them. This prevents a defensible location-level violation-utility objective.

Proxy cost versus realized overhead. Event-check totals do not include JVM, instrumentation, shared monitoring work, or interactions among selected groups. Additive proxy budgets must not be interpreted as runtime savings.

Controllability. An inferred first-event location and specification group may not correspond to an independently switchable creation site. The monitoring implementation must be inspected before deploying selected sets.

Evaluation scope. The current offline analysis is in-sample and lacks a held-out execution split. The artifact timings are single runs, and three projects lack a complete ValgT/TraceMOP pair. We therefore cannot claim generalization, memory savings, or superiority over Valg.

6. Reproducibility

The repository now separates the two evidence streams. src/run_experiments.py and results/offline contain the selection algorithms and proxy-budget analysis. src/valg_baselines contains the official-artifact runner, summarizer, plotting code, and exact invocation guide. configs/valg_baselines records repository commits; results/valg_baselines contains compact raw outputs, derived summaries, and figures; data/valg_profiles contains algorithm-ready CSV profiles extracted from successful full TraceMOP runs.

7. Schedule and Deliverables

Oct. 19-25: validate group IDs and extraction semantics; curate workloads. **Oct. 26-Nov. 1:** implement or test enable/disable controls and collect per-site violation associations. **Nov. 2-8:** profile representative configurations with repeated runtime and memory measurements. **Nov. 9-15:** compare budgeted policies and Valg at similar measured overhead on held-out executions. **Nov. 16-end:** robustness analyses, possible revision-transfer/exploration study, and final report.

Shiyun maintains the selection/evaluation pipeline; Gengkun coordinates Valg profiling and instrumentation. If independently switchable monitors or incremental timing prove infeasible, the final work will explicitly narrow its claims to an offline proxy-cost benchmark instead of treating proxy budgets as runtime savings.